Olist SQL Formatting and Safety Guide

---

1. Overview

This document defines best practices for writing SQL queries in the Olist database.

It ensures:

- safe query execution
- consistent formatting
- correct use of tables and joins
- compatibility with the system's SQL validation rules

---

2. Allowed SQL Operations

Allowed

- SELECT
- FROM
- WHERE
- GROUP BY
- ORDER BY
- JOIN
- LIMIT

Not Allowed

The following operations are strictly prohibited:

- DROP
- DELETE
- UPDATE
- INSERT

- ALTER
- CREATE

Rule:
Only **read-only queries (SELECT)** are allowed.

---

3. Query Structure Guidelines

Basic Query Template

```
SELECT column1, column2
FROM table_name
WHERE condition
GROUP BY column
ORDER BY column DESC
LIMIT 10;
```

Example 1 — Count Orders

```
SELECT COUNT(DISTINCT order_id) AS total_orders
FROM orders;
```

Example 2 — Revenue by Category

```
SELECT p.product_category_name, SUM(oi.price) AS total_revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_category_name
ORDER BY total_revenue DESC;
```

Example 3 — Payment Types

```
SELECT payment_type, COUNT(*) AS count
FROM payments
GROUP BY payment_type
ORDER BY count DESC;
```

---

4. Join Guidelines

Common Joins

- Orders ↔ Order Items

orders.order_id = order_items.order_id

- Order Items ↔ Products

order_items.product_id = products.product_id

- Orders ↔ Customers

orders.customer_id = customers.customer_id

- Orders ↔ Payments

orders.order_id = payments.order_id

- Orders ↔ Reviews

orders.order_id = reviews.order_id

Best Practices

- Always use **explicit JOIN syntax**
- Avoid Cartesian joins (missing ON conditions)
- Use table aliases for readability:

FROM order_items oi
JOIN products p ON oi.product_id = p.product_id

---

5. Aggregation Rules

Common Aggregations

- COUNT(*) → number of rows
- COUNT(DISTINCT column) → unique count
- SUM(price) → total revenue

- AVG(review_score) → average rating

Important Notes

- Always use GROUP BY when aggregating by category
- Ensure aggregation level is correct (order vs item)

---

6. Filtering Guidelines

Examples

WHERE order_status = 'delivered'

WHERE order_purchase_timestamp >= '2018-01-01'

Best Practices

- Always specify conditions clearly
- Avoid ambiguous filters
- Use date filters when analyzing trends

---

7. Safety Rules

- Only SELECT queries are allowed
- Never modify database data
- Validate queries before execution
- Avoid querying unnecessary columns

---

8. Error Prevention Tips

- Ensure column names exist before using them
- Use get_schema() when unsure
- Check join keys carefully

- Avoid mixing aggregated and non-aggregated columns

---

9. Notes

- Always use get_schema() if table or column names are uncertain
- Always validate SQL before executing
- Never assume column names
- Prefer simple, readable queries over complex ones